Projeto Final — Etapa 2

Operação, interface e desempenho

Universidade Católica de Brasília — Bacharelado em Engenharia de Software

Unidade curricular	Laboratório de Banco de Dados — GPE17M40053
Professor	Samuel Novais Moura Júnior
Ano/semestre	2026/2
Tema	Cooperativa Agrícola — safras, lotes rastreáveis, associados com histórico de entregas e prestação de contas
SGBD	MySQL 8.0.46
Repositório	https://github.com/arthurgs645-ctrl/projeto-final-cooperativa-agricola

Equipe

- Artur Gabriel Queiroz de Morais
- Arthur Gonçalves Silva
- Arthur Marques Cipriano
- Arthur Mélo Mesquita Mitrovich
- Fabrício Nogueira de Brito

---

1. Resposta à devolutiva da Etapa 1

A devolutiva da Etapa 1 encerrou com a orientação *"incorpore esta devolutiva ao relatório final"*. Esta seção responde cada apontamento, dizendo o que foi feito, onde verificar, e — quando foi o caso — o que **não** foi feito e por quê.

C1 — Modelo conceitual

> *"na Etapa 2, aumente o diagrama e destaque os papéis de INDICA"*

Parcialmente atendido. O diagrama foi ampliado e o autorrelacionamento `INDICA` recebeu rótulo explícito nos dois papéis (*indicador* / *indicado*), que antes apareciam sem nome. O arquivo-fonte editável continua em `docs/mer-conceitual.drawio`.

Registro honesto: esta foi a correção de menor profundidade das cinco. A equipe priorizou os apontamentos de C3 e C5, que tinham efeito direto sobre o banco e a documentação verificável. Se o diagrama ainda estiver abaixo do esperado, é consequência dessa escolha de prioridade, não de desatenção ao apontamento.

C2 — Modelo lógico

> *"demonstre a BCNF com as DFs (não apenas afirme) e discuta se o saldo derivado é uma redundância deliberada. Compare também, por escrito, a estratégia de PF/PJ com as alternativas."*

Atendido. Ver seção 9 deste relatório, que traz:

- as dependências funcionais de cada relação, escritas, e a verificação de BCNF relação por relação;
- a discussão do `saldo` como redundância derivada deliberada, com o custo e o benefício da escolha;
- a comparação da estratégia de especialização PF/PJ com as duas alternativas (tabela única com colunas nulas, e tabelas separadas sem supertipo).

C3 — Script físico (DDL)

> *"considere reforçar no banco o que hoje fica só em consulta (autoindicação, totalidade e disjunção PF/PJ, cronologia do histórico), e justifique cada ação referencial"*

Atendido, e este foi o apontamento de maior impacto. Quatro regras que a Etapa 1 verificava apenas por consulta passaram a ser impostas pelo próprio banco, em `sql/09_reforco_restricoes.sql`. As dez tentativas de violação registradas em `experimentos/resultados/C3_reforco_restricoes.txt` foram todas recusadas pelo servidor, e as operações legítimas continuam passando.

Detalhe que vale destacar, porque mudou a solução: a autoindicação **não pôde** ser implementada por `CHECK`. A tentativa retornou

```
ERROR 3823 (HY000): Column 'id_associado_indicador' cannot be used in a check
constraint 'ck_associado_sem_autoindicacao': needed in a foreign key
constraint 'fk_associado_indicador' referential action.
```

O motivo é que a Etapa 1 declarou `fk_associado_indicador` com `ON DELETE SET NULL`. Como essa ação referencial pode alterar a coluna por iniciativa do servidor, o MySQL recusa qualquer `CHECK` sobre ela — do contrário, apagar um associado poderia deixar uma linha violando a restrição sem que ninguém tivesse escrito nela. A regra foi então implementada por gatilho.

Isso ilustra o próprio pedido de "justificar cada ação referencial": a escolha de `ON DELETE SET NULL` não foi neutra. Ela comprou a comodidade de não perder o associado quando seu indicador sai do quadro, e pagou com a impossibilidade de usar `CHECK` naquela coluna. A justificativa de todas as ações referenciais está na seção 9.4.

Sobre **totalidade**, uma ressalva: ela não é imposta por gatilho, e isso é deliberado. Exigir a linha filha no momento do `INSERT` do pai tornaria impossível inserir o associado, já que a filha depende da chave do pai que ainda não existe. O modelo relacional sem `ASSERTION` não resolve isso de forma limpa. Em vez de fingir que está resolvido, criamos a visão `vw_especializacao_irregular`, que lista a qualquer momento os associados sem a especialização correspondente. No banco atual ela devolve zero linhas.

C4 — Carga de dados

> *"os dados parecem gerados em série (CPFs sequenciais, 'Empresa Rural N', nomes repetidos, safras sobrepostas e de 14 meses, poucas safras com prestação); torne-os mais realistas e*

> *variados"*

Quanto ao realismo: não atendido nesta entrega. É o apontamento que ficou de fora, e a equipe prefere declará-lo a omiti-lo. Os dados continuam com as características que a devolutiva apontou.

Mas a tentativa de reconstruir o banco do zero revelou um defeito mais grave, que foi corrigido. O `02_carga.sql` entregue na Etapa 1 **não executava em banco limpo**:

```
ERROR 1136 (21S01) at line 6: Column count doesn't match value count at row 1
```

O `INSERT` de `associado` declarava seis colunas e cada linha trazia cinco valores. Além da contagem, havia troca de posição: os valores estavam na ordem `(id, nome, matricula, status, indicador)`, enquanto a lista declarava `(id, matricula, nome, tipo_pessoa, status, indicador)`. A coluna `tipo_pessoa`, que é `NOT NULL` sem `DEFAULT`, simplesmente não aparecia.

O defeito passou despercebido porque **ninguém reconstruiu o banco do zero** — o ambiente de desenvolvimento já estava povoado por uma versão anterior do script, e todas as verificações rodaram sobre esse banco pronto. É exatamente o item do Anexo B que diz *"o banco foi apagado e o script executou do início ao fim sem erro"*, e que não foi cumprido de fato.

Correção aplicada: o bloco foi reescrito preservando os dados originais, com as colunas na ordem correta e `tipo_pessoa` deduzido dos blocos `associado_pf` (ids 1–30) e `associado_pj` (ids 31–40) do próprio arquivo. A correção está marcada em comentário no cabeçalho do script.

Verificação: o banco foi apagado por completo (`DROP DATABASE`) e os nove scripts foram executados em sequência. Resultado em `experimentos/resultados/B8_reconstrucao_limpa.txt`:

Script	Resultado
`01_ddl.sql`	OK (0,7 s)
`02_carga.sql`	OK (0,1 s)
`03_consultas.sql`	OK (0,1 s)
`04_views.sql`	OK (0,1 s)
`05_triggers_auditoria.sql`	OK (0,1 s)
`06_seguranca_perfis.sql`	OK (0,2 s)
`09_reforco_restricoes.sql`	OK (0,1 s)
`07_carga_volume.sql`	OK (14 s)
`08_otimizacao_indices.sql`	OK (3,1 s)

Zero erros em nove scripts. Toda a bateria de testes — 36 da aplicação, 11 de perfil, 10 de restrição, mais os experimentos de concorrência, injeção e desempenho — foi reexecutada contra esse banco reconstruído, e os números deste relatório vêm dessa execução.

Houve, porém, uma alteração de volume com outra finalidade: `sql/07_carga_volume.sql` eleva a tabela `entrega` a 200.200 linhas **para o estudo de desempenho do B4**. Com as 160 entregas originais, qualquer plano de execução é rápido e o otimizador escolhe varredura completa independentemente de índice — medir ganho ali produziria números sem significado. O script é reversível (`DELETE FROM`

`entrega WHERE id_entrega > 160`) e as linhas sintéticas respeitam as mesmas chaves e domínios, inclusive a RN20 recém-imposta por gatilho.

Essa carga de volume **não responde** ao apontamento de realismo: ela resolve um problema de medição, não de qualidade dos dados. O realismo da carga segue pendente.

Versionamento — item que faltava

O enunciado exige, nos Requisitos Técnicos, *"Git, em repositório remoto acessível ao professor"*, e a avaliação individual considera *"o histórico de commits no repositório"* como uma de suas três fontes.

A Etapa 1 foi entregue sem repositório. O pacote trazia um `.gitignore`, o que indica que o versionamento chegou a ser planejado, mas nenhum repositório foi criado — portanto uma das fontes de avaliação individual simplesmente não existia.

Corrigido nesta etapa:

https://github.com/arthurgs645-ctrl/projeto-final-cooperativa-agricola

O repositório contém as duas etapas, com o histórico organizado por frente de trabalho, e nenhuma credencial versionada (o `.env` está no `.gitignore`; o que se versiona é o `.env.example`).

Ressalva honesta sobre o histórico: os commits foram criados nesta data, ao montar o repositório, e não acompanham o desenvolvimento ao longo do semestre. Eles descrevem corretamente *o que* foi feito em cada frente, mas não servem como medida de *quando* nem de distribuição de esforço entre os integrantes. Registramos isso para que o histórico não seja lido como algo que ele não é.

C5 — Documentação

> *"corrija o código da turma na capa e no A1, inclua todos os integrantes, atualize a árvore do README e complete o dicionário de dados no formato do Anexo A (tipos, restrições)"*

Atendido integralmente.

Apontamento	O que foi feito
Código da turma	Era `GPE17M40083` na capa e no A1; corrigido para **`GPE17M40053`**, conforme o AVA
Integrantes	Os cinco nomes constam da capa deste relatório, do README e do cabeçalho de cada script SQL
Árvore do README	Reescrita, refletindo a estrutura real da Etapa 2
Dicionário de dados	Regerado no formato do Anexo A, com tipo, obrigatoriedade, restrições nomeadas e a regra de negócio por atributo — 15 tabelas, 122 colunas. Ver `docs/A3_dicionario_dados.md`

O dicionário passou a ser **gerado** a partir do `information_schema` por `experimentos/gera_dicionario.py`, e não digitado à mão. Assim ele não pode divergir do esquema real: se uma coluna mudar de tipo, basta executar o script de novo.

A declaração de uso de IA foi mantida e atualizada (seção 10).

---

2. B1 — Aplicação com interface

Aplicação web em **Python com Flask**, conectada ao banco construído na Etapa 1. A interface é deliberadamente simples: o que ela precisa revelar é o comportamento do banco, não sofisticação visual.

Cobertura funcional

Requisito	Onde está	Evidência
CRUD de 4 entidades	`associado`, `propriedade`, `cultura`, `lote`	8 testes em `B1_teste_aplicacao.txt`
2 consultas parametrizadas com junção e agregação	telas *Consulta: produção* e *Consulta: associados*	2 testes
Relatório por visão	tela *Relatório*, lê `vw_relatorio_safra_associado`	1 teste
Transação multitabela	tela *Fechar prestação*	4 testes
Erro de restrição tratado	todas as telas de cadastro	5 testes

Consulta 1 — produção por cultura. Junta `entrega`, `lote`, `cultura` e `safra`, agrega por cultura e safra, aplica `HAVING` sobre a soma. Cinco filtros do usuário: período (duas datas), situação, nome da cultura e quantidade mínima.

Consulta 2 — desempenho por associado. Junta `associado`, `propriedade`, `lote` e `entrega` com `LEFT JOIN`, para não perder o associado que ainda não entregou nada. Quatro filtros.

Relatório. A tela não monta junção nenhuma: faz `SELECT` na visão `vw_relatorio_safra_associado`. A regra de consolidação mora no banco, onde pode ser reusada por outras ferramentas.

Tratamento de erro de restrição

A aplicação nunca exibe a exceção do driver. Além de incompreensível para quem opera a cooperativa, o texto cru revela nomes de tabela e estrutura interna, que é informação útil para quem tenta atacar o sistema.

A tradução é feita em `app/db.py` e se apoia na convenção de nomes adotada na Etapa 1 (`pk_` / `uq_` / `fk_` / `ck_`): o nome da restrição que aparece na mensagem do servidor é procurado num dicionário e devolvido como frase de domínio. Exemplos medidos:

Violação	O que o usuário vê
`uq_cultura_nome`	"Já existe uma cultura com esse nome."
`ck_lote_quantidade`	"A quantidade planejada precisa ser maior que zero."
`ck_prestacao_descontos`	"Os descontos não podem superar o valor bruto."
FK com `RESTRICT`	"Não dá para remover: existem registros dependentes ligados a este. Remova ou transfira os dependentes antes."
Campo obrigatório vazio	"Um campo obrigatório ficou em branco."

Um achado do teste vale registro: esperávamos o erro **1451** (`ER_ROW_IS_REFERENCED_2`) ao remover um associado com propriedades, e o MySQL 8.0 devolveu **1217** (`ER_ROW_IS_REFERENCED`). Os dois códigos descrevem a mesma situação e qual deles aparece depende de como a violação é detectada. O mapeamento trata ambos.

Resultado do teste de ponta a ponta

`experimentos/teste_aplicacao.py` percorre o checklist do Anexo C pela interface HTTP real, como um usuário faria pelo navegador.

36 verificações, 36 aprovadas, 0 falhas.

---

3. B2 — Camada de acesso a dados

Consultas parametrizadas

Todo valor vindo do usuário viaja como **parâmetro**, nunca concatenado no texto do comando. A única coisa montada dinamicamente é a lista de colunas de um `INSERT` / `UPDATE`, e os nomes de coluna vêm de uma estrutura fechada definida no código (`ENTIDADES`, em `app/db.py`) — um campo extra injetado no formulário pelo navegador é descartado antes de chegar ao SQL.

Vale destacar uma decisão que evita concatenação em filtro opcional. Em vez de acrescentar a cláusula só quando o campo está preenchido, o próprio comando trata o campo vazio:

```
AND (%s = '' OR c.nome LIKE CONCAT('%', %s, '%'))
```

O texto do SQL nunca muda, independentemente de quais filtros o usuário preencheu.

Demonstração de injeção de SQL

`experimentos/injecao_sql.py` executa a **mesma entrada maliciosa** contra duas implementações, no banco real: uma versão vulnerável (concatenação) e a versão que a aplicação usa (parametrizada). A versão vulnerável existe apenas nesse arquivo de demonstração, para que a diferença seja medida e não apenas afirmada.

Controle. Com a entrada legítima `Ana Cardoso`, as duas versões devolvem as mesmas 2 linhas.

Ataque 1 — tautologia. Entrada: `' OR '1'='1`

	SQL que chegou ao banco	Resultado
Vulnerável	`... WHERE nome = '' OR '1'='1'`	**40 linhas — a tabela inteira vazou**
Parametrizada	`... WHERE nome = %s`	**0 linhas**

Ataque 2 — UNION para extrair CPF. Entrada: `' UNION SELECT 1, cpf, 1 FROM associado_pf --`

Na versão vulnerável o comando chegou montado ao servidor, mas foi barrado por **outra** camada: a conta `app_leitura` não tem `SELECT` em `associado_pf` (erro 1142). Na versão parametrizada, o texto foi tratado como um nome a procurar e devolveu 0 linhas.

Esse caso é o mais instrutivo dos três, porque mostra as duas defesas operando em profundidade: mesmo que a parametrização falhasse, o menor privilégio impediria a leitura do CPF.

Ataque 3 — comentário. Entrada: `Ana Cardoso' --` — truncaria o restante do comando na versão vulnerável; na parametrizada, procura literalmente um associado com esse nome e devolve 0 linhas.

Por que a parametrização funciona. A diferença não está em filtrar ou escapar caracteres, e sim em *como* o comando chega ao servidor. O texto do SQL é enviado e compilado uma vez, com marcadores no lugar dos valores; os valores viajam depois, por um canal separado do protocolo. Quando o servidor recebe `' OR '1'='1`, ele já terminou de decidir o que o comando faz — aquele texto só pode ser conteúdo do parâmetro, jamais sintaxe.

Por isso a aplicação não tenta remover aspas nem palavras como `DROP`: abordagem de lista de bloqueio falha sempre que alguém encontra uma codificação não prevista. Parametrizar elimina a classe inteira do problema.

Credenciais fora do código

Nenhuma senha aparece no código-fonte. Tudo é lido de variáveis de ambiente, carregadas de um `.env` que está no `.gitignore`. O arquivo versionado é o `.env.example`, só com os nomes das variáveis. Se uma variável obrigatória faltar, a aplicação falha na inicialização com mensagem clara, em vez de tentar conectar com um valor padrão inseguro.

---

4. B3 — Transações e controle de concorrência

A operação transacional

Fechar a prestação de contas de um associado numa safra. Altera duas tabelas: `prestacao_contas` e `movimento_financeiro`.

Passos dentro da transação:

1. `START TRANSACTION`
2. apura o bruto somando as entregas `RECEBIDA` do associado na safra
3. calcula desconto (taxa da cooperativa) e saldo
4. `INSERT ... ON DUPLICATE KEY UPDATE` em `prestacao_contas` *(tabela 1)*
5. `INSERT ... ON DUPLICATE KEY UPDATE` em `movimento_financeiro`, lançando o crédito *(tabela 2)*
6. `UPDATE prestacao_contas SET status = 'FECHADA'`
7. `COMMIT`

Por que precisa ser atômico. Se o passo 5 falhasse depois do passo 4, a cooperativa teria uma prestação fechada sem o lançamento financeiro que a paga — o associado apareceria como quitado sem ter recebido. Os dois comandos descrevem um único fato do negócio e têm que valer juntos ou nenhum.

Execução real registrada:

```
START TRANSACTION
SELECT apuracao -> bruto = 52751785.73 (4009 entregas recebidas)
INSERT/UPDATE prestacao_contas -> bruto 52751785.73, desconto 2637589.29, saldo 50114196.44
INSERT movimento_financeiro -> CREDITO 50114196.44 (referencia CRED-81-1-1)
UPDATE prestacao_contas -> status = FECHADA
COMMIT
```

Falha provocada no meio da transação

A aplicação tem um botão que injeta um desconto maior que o bruto **depois** que a primeira tabela já foi escrita, violando `ck_prestacao_descontos`:

```
START TRANSACTION
SELECT apuracao -> bruto = 52751785.73 (4009 entregas recebidas)
FALHA INJETADA: desconto forcado para 79127678.60, maior que o bruto 52751785.73
ROLLBACK
motivo: Os descontos nao podem superar o valor bruto.
```

A tela mostra o estado das duas tabelas antes e depois, lado a lado. No caminho com falha as duas linhas ficam idênticas: nenhuma prestação gravada pela metade, nenhum crédito órfão.

Um erro que a equipe cometeu e vale registrar. A primeira versão calculava os valores com `float`. A transação passou a falhar em `ck_prestacao_saldo`, que exige `saldo = total_bruto - total_descontos`. As colunas são `DECIMAL(14,2)` e o `CHECK` é avaliado em aritmética decimal exata; calculando em ponto flutuante, o desconto de 5% sobre 52.751.785,73 chegava ao banco com um resíduo binário invisível, e a subtração feita pelo MySQL não batia com o saldo enviado. A correção foi usar `decimal.Decimal` em toda a conta monetária. Dinheiro não se calcula em ponto flutuante — e foi a restrição do banco que denunciou o erro.

Experimento de concorrência

Executado com duas conexões simultâneas ao banco real. Registro completo em `experimentos/resultados/B3_concorrencia.txt`.

Ambiente: MySQL 8.0.46, isolamento `REPEATABLE-READ` (padrão do InnoDB), `autocommit = 0`.

Cenário 1 — mesma linha. A sessão A abre transação, altera a entrega 1 e segura por 5 segundos. A sessão B tenta alterar a mesma linha.

Estado interno observado durante a espera:

```
transacao esperando  : 18917 (thread 232)
comando dela         : UPDATE entrega SET qualidade = 'C' WHERE id_entrega = 1
bloqueada pela trx   : 18916 (thread 231)
```

B não recebeu erro nem leu valor desatualizado: **ficou em espera por 5,03 s**, exatamente o tempo que A segurou a transação, e só então aplicou sua alteração sobre o valor já atualizado. Se A nunca confirmasse, B esperaria até `innodb_lock_wait_timeout` (50 s por padrão) e receberia o erro 1205, que a aplicação traduz em mensagem.

Cenário 2 — isolamento. A lê a entrega 1 (`qualidade = C`). B altera para `A` e confirma. A lê de novo, **na mesma transação**, e continua vendo `C`. Só depois do próprio `COMMIT` é que A passa a enxergar `A`.

É `REPEATABLE READ` funcionando: A trabalha sobre uma fotografia consistente tirada na primeira leitura (MVCC).

Cenário 3 — deadlock. A trava a linha 1 e pede a 2; B trava a 2 e pede a 1.

```
Sessao A: travou linha 1 -> ficou esperando
          desfecho: ABORTADA pelo servidor - erro 1213: Deadlock found when trying to get lock
Sessao B: travou linha 2 -> travou linha 1
          desfecho: COMMIT
```

Diferente do cenário 1, aqui ninguém poderia terminar esperando: cada uma segurava o que a outra precisava. O InnoDB detectou o ciclo e escolheu uma vítima, abortando-a por completo — não resta estado pela metade.

Justificativa do nível de isolamento

A aplicação mantém **REPEATABLE READ**, padrão do InnoDB, por três razões ligadas ao que este sistema faz:

1. **A transação central soma e depois grava.** Em `READ COMMITTED`, uma entrega confirmada por outra sessão entre a apuração e a gravação mudaria o total sem que a transação percebesse, e a prestação seria gravada com um valor que já não corresponde às entregas. `REPEATABLE READ` garante que a soma e a gravação enxerguem a mesma fotografia.
1. **Não precisamos de SERIALIZABLE.** Ele acrescentaria bloqueio de intervalo em toda leitura, encarecendo as consultas do pátio — que são frequentes — para resolver um problema que não temos: `uq_prestacao_associado_safra` já impede dois fechamentos concorrentes para o mesmo par associado/safra.
1. **Em troca, aceitamos bloqueio e possibilidade de deadlock**, como os cenários 1 e 3 mostraram. A aplicação trata os dois: os erros 1205 e 1213 viram mensagem orientando a repetir a operação.

---

5. B4 — Otimização de consultas

Volume no momento da medição: 200.200 entregas, 120 lotes, 40 propriedades, 40 associados, 81 prestações.

Método. Para cada consulta, o índice é removido para obter o estado "antes". Mede-se 5 execuções após uma de aquecimento, registrando média e melhor tempo. O tempo de parede inclui o custo de abrir a conexão do cliente `mysql`, constante entre as medições — por isso o `EXPLAIN ANALYZE` também é registrado, mostrando o tempo do servidor isolado.

Consulta 1 — janela de recebimento

```
SELECT COUNT(*), SUM(quantidade), SUM(quantidade * preco_unitario_kg)
  FROM entrega
 WHERE data_entrega BETWEEN '2024-03-01' AND '2024-06-30'
   AND status = 'RECEBIDA';
```

É a consulta que abre a tela de conferência diária do pátio, a mais executada do sistema.

	Plano	Tempo de parede	Tempo do servidor
Antes	`Index range scan` em `idx_entrega_data`, filtro residual de status	54,6 ms	17,3 ms
Depois	`Covering index range scan` em `idx_entrega_data_status_valor`	45,0 ms	**7,58 ms**

Leitura do plano, antes:

```
type=range | key=idx_entrega_data | key_len=3 | rows=4815 | filtered=50.00
Extra: Using index condition; Using where
```

- `type = range` — acesso por faixa de índice, não varredura. O servidor percorre só o trecho do índice entre as duas datas.
- `key_len = 3` — três bytes de chave, que é o tamanho de um `DATE`. Confirma que **apenas `data_entrega`** participou da busca; o índice não tem mais nada a oferecer.
- `filtered = 50.00` — o otimizador estima que metade das linhas lidas sobreviverá aos filtros restantes. Esse "restante" é o `status`, que não está no índice.
- `Using index condition` — parte do filtro foi empurrada para o motor de armazenamento (*index condition pushdown*), evitando devolver linhas que já se sabe que serão descartadas.
- `Using where` — ainda sobra filtragem feita no servidor, acima do índice.

Em resumo: lê 4.815 entradas do índice, **vai à tabela buscar cada linha correspondente** e só então descarta as que não são `RECEBIDA`. O custo está nos 4.815 saltos à tabela.

Leitura do plano, depois:

```
type=range | key=idx_entrega_data_status_valor | key_len=85 | rows=8652 | filtered=50.00
Extra: Using where; Using index
```

- `key_len = 85` — subiu de 3 para 85 bytes: `data_entrega` (3) mais `status` (82). As **duas** colunas agora participam da busca.
- `Using index`, sem `index condition` — este é o ponto central: o índice é **cobridor**. Todas as colunas de que a consulta precisa estão nele, então o plano **não toca a tabela nenhuma vez**.

Um detalhe que parece contradição e vale explicar: `rows` **aumentou** de 4.815 para 8.652, e mesmo assim o tempo caiu de 17,3 ms para 7,58 ms. Não é contradição, é mudança de unidade de custo. `rows` conta entradas de índice percorridas, não acessos a disco. Percorrer 8.652 entradas sequenciais de um índice cobridor é muito mais barato que percorrer 4.815 e fazer 4.815 saltos aleatórios à tabela. **Ler `rows` como se fosse "trabalho" é o erro mais comum ao interpretar um `EXPLAIN`.**

Índice criado: `(data_entrega, status, quantidade, preco_unitario_kg)`.

Ordem das colunas. `data_entrega` primeiro por ser o predicado de **faixa**, que define o intervalo a percorrer; `status` em seguida, para filtrar dentro da faixa. Invertendo, o índice só seria usado até a igualdade de status e a faixa de data viraria filtro residual. `quantidade` e `preco_unitario_kg` entram ao

final para tornar o índice **cobridor**: o plano resolve a agregação sem nenhum acesso à tabela base, o que o `EXPLAIN` confirma com `Using index`.

Ganho real no servidor: **56%**.

Consulta 2 — contagem por status (a hipótese da equipe estava errada)

```
SELECT COUNT(*) FROM entrega WHERE status = 'RECEBIDA';
```

Nossa previsão era que o índice seria inútil. O raciocínio parecia sólido: 160.355 das 200.200 linhas têm `status = 'RECEBIDA'`, ou seja, 80% da tabela. Um índice que aponta para 80% das linhas normalmente é ignorado pelo otimizador, porque o acesso aleatório resultante sairia mais caro que ler tudo em sequência.

A medição desmentiu a previsão:

	Plano	Tempo de parede	Tempo do servidor
Antes	`Covering index skip scan`	233,5 ms	301 ms
Depois	`Covering index lookup` em `idx_entrega_status`	118,5 ms	**153 ms**

Queda de 49% no tempo de parede e 49% no tempo de servidor.

Leitura do plano, antes:

```
type=range | key=idx_entrega_data_status_valor | key_len=85 | rows=66599 | filtered=100.00
Extra: Using where; Using index for skip scan
```

- `Using index for skip scan` — recurso do MySQL 8.0 que merece explicação. O índice disponível começa por `data_entrega`, coluna que **esta** consulta não filtra. Normalmente isso tornaria o índice inútil, porque um índice composto só pode ser percorrido da esquerda para a direita. O *skip scan* contorna: o servidor percorre os valores distintos do primeiro nível e, para cada um, faz uma busca pelo segundo. É melhor que varredura de tabela, mas paga o preço de repetir a busca para cada valor de data.
- `filtered = 100.00` — tudo que o índice devolve já satisfaz o filtro.

Leitura do plano, depois:

```
type=ref | key=idx_entrega_status | key_len=82 | ref=const | rows=99899 | filtered=100.00
Extra: Using index
```

- `type = ref` — mudou de `range` para `ref`, que é acesso por **igualdade**. Melhor que faixa: o servidor vai direto ao ponto do índice onde começam as linhas com aquele valor.
- `ref = const` — o valor comparado é uma constante (`'RECEBIDA'`), conhecida antes da execução.
- `Using index` — cobridor, sem acesso à tabela.
- Note que `rows = 99.899` é uma estimativa **grosseira**: a execução real percorreu 160.355 linhas. Isso é esperado e vale registrar — o otimizador trabalha com histogramas e cardinalidades aproximadas, não com a contagem exata. A decisão de plano é tomada sobre estimativa, não sobre verdade.

Por que erramos. O ganho não vem de seletividade, vem de **largura**. A consulta é `COUNT(*)` com filtro apenas em `status`, então o índice é cobridor: o MySQL responde percorrendo só a entrada do índice secundário — `status` mais o ponteiro da chave primária — sem tocar na tabela. Como a linha completa de `entrega` tem oito colunas, percorrer o índice estreito lê muito menos páginas do que percorrer o índice agrupado.

A lição: seletividade baixa afasta o índice quando ele obriga a voltar na tabela. Quando o índice cobre a consulta inteira, ele continua valendo a pena mesmo com seletividade ruim. O índice foi **mantido** por causa do resultado medido, não da previsão.

Ressalva metodológica. No estado "antes" desta consulta, o índice da Consulta 1 ainda estava presente, e o otimizador o usou em *skip scan*. Portanto a linha de base não foi uma varredura limpa de tabela, e o ganho de 50% compara duas estratégias de índice, não índice contra ausência de índice. Registramos isso porque muda a interpretação do número.

Consulta 3 — classificação por qualidade e faixa de preço

```
SELECT qualidade, COUNT(*), AVG(preco_unitario_kg)
  FROM entrega
 WHERE qualidade = 'A' AND preco_unitario_kg BETWEEN 5.00 AND 9.00
 GROUP BY qualidade;
```

	Plano	Tempo de parede	Tempo do servidor
Antes	`Table scan` — 200.200 linhas lidas para devolver 29.720	144,1 ms	117 ms
Depois	`Covering index range scan` em `idx_entrega_qualidade_preco`	59,8 ms	**36,5 ms**

Leitura do plano, antes:

```
type=ALL | key=NULL | rows=199798 | filtered=1.11
Extra: Using where
```

- `type = ALL` — **varredura completa de tabela**, o pior tipo de acesso. O servidor lê todas as linhas do índice agrupado.
- `key = NULL` — nenhum índice utilizável. A coluna `qualidade` não é a primeira de nenhum índice existente.
- `filtered = 1.11` — **este é o número mais eloquente do estudo inteiro.** O otimizador estima que apenas 1,11% das linhas lidas sobreviverão ao filtro: lê 199.798 para aproveitar cerca de 2.200. É a assinatura clássica de índice faltando — percorrer tudo para descartar quase tudo.

Leitura do plano, depois:

```
type=range | key=idx_entrega_qualidade_preco | key_len=9 | rows=54330 | filtered=100.00
Extra: Using where; Using index
```

- `type` saiu de `ALL` para `range` — deixou de varrer a tabela e passou a percorrer só o trecho do índice correspondente a `qualidade = 'A'` com preço na faixa pedida.

- `key_len = 9` — as duas colunas do índice participam da busca.
- `filtered` saltou de **1,11% para 100%** — o salto é a medida exata do que o índice resolveu. Antes, 98,9% do trabalho de leitura era desperdiçado em linhas que seriam descartadas; agora nada é lido à toa.
- `Using index` — cobridor: `qualidade` e `preco_unitario_kg` são as únicas colunas de que a consulta precisa, e ambas estão no índice.

Índice criado: `(qualidade, preco_unitario_kg)`.

Ordem das colunas. `qualidade` primeiro por ser predicado de **igualdade**, `preco_unitario_kg` depois por ser predicado de **faixa**. É a regra geral: igualdade antes de faixa. Com a ordem invertida, o índice seria percorrido pela faixa de preço e a qualidade viraria filtro linha a linha.

Ganho real no servidor: **69%**.

O custo dos índices

Medição em `information_schema.tables` depois do estudo:

Tabela	Dados	Índices	Total
`entrega`	11,52 MB	**25,58 MB**	37,09 MB

Os índices de `entrega` ocupam **mais que o dobro dos dados**. Isso é o outro lado da conta que raramente aparece quando se discute otimização: cada índice é pago três vezes — em disco, em memória de cache, e em tempo de `INSERT` / `UPDATE`, já que toda escrita precisa manter todas as árvores. Em uma tabela de alta taxa de inserção como `entrega`, que recebe toda a produção do pátio, essa conta é relevante e justificaria revisar se os três índices se pagam em produção.

---

6. B5 — Segurança e controle de acesso

Três perfis com menor privilégio

Perfil	Função na cooperativa	Pode	Não pode
`coop_leitura`	conselho fiscal, auditoria, análise	ler a operação inteira	escrever qualquer coisa; ver CPF/CNPJ; ler as tabelas financeiras por linha
`coop_operacional`	secretaria e pátio	CRUD em cadastros, lotes e entregas	tocar em qualquer tabela financeira; apagar entrega
`coop_financeiro`	tesouraria	fechar prestação, lançar movimento, ler auditoria	alterar entrega, lote ou cadastros

Duas decisões merecem explicação.

Ausência de `DELETE` em `entrega` para o perfil operacional. Estorno de entrega é feito mudando o status para `CANCELADA`, nunca apagando a linha: a entrega é o documento que origina o pagamento e precisa permanecer rastreável. É decisão de modelagem, não esquecimento.

O perfil de leitura enxerga o relatório financeiro consolidado sem ter `SELECT` em `prestacao_contas`. Isso funciona porque, por padrão, a visão do MySQL é `SQL SECURITY DEFINER` — executa com o privilégio de quem a criou, não de quem a consulta. É exatamente esse mecanismo que permite liberar o agregado sem liberar a linha.

Demonstração de REVOKE

Durante o desenvolvimento, o perfil financeiro havia recebido `UPDATE` em `entrega` para "corrigir preços". Isso viola separação de funções: quem fecha o pagamento não deve poder alterar o documento que origina o valor. O script registra o estado anterior e o corrige:

```
GRANT  UPDATE ON cooperativa_agricola.entrega TO coop_financeiro;
REVOKE UPDATE ON cooperativa_agricola.entrega FROM coop_financeiro;
```

Visões de restrição

- **Coluna:** `vw_associado_publico` expõe o documento mascarado (`..000-`) **em vez de CPF/CNPJ. O perfil de leitura recebe `SELECT` nela e** não** nas tabelas `associado_pf` / `associado_pj`.
- **Linha:** `vw_lote_em_operacao` esconde safras e lotes encerrados de quem só precisa do que está em andamento.

Teste com login real

`experimentos/resultados/B5_teste_perfis.txt` — onze operações, cada uma conectando com a conta real do perfil. **11 de 11 conferem.**

Perfil	Operação	Esperado	Obtido
leitura	consultar visão mascarada	permitido	permitido
leitura	ler CPF real na tabela base	negado	**negado** (1142)
leitura	ler `prestacao_contas` por linha	negado	**negado** (1142)
leitura	ler o saldo pela visão consolidada	permitido	permitido
leitura	inserir entrega	negado	**negado** (1142)
operacional	consultar lotes	permitido	permitido
operacional	ler prestação de contas	negado	**negado** (1142)
operacional	apagar entrega	negado	**negado** (1142)
financeiro	consultar prestação	permitido	permitido
financeiro	alterar preço de entrega	negado	**negado** (após `REVOKE`)
financeiro	alterar cadastro de associado	negado	**negado** (1142)

As duas linhas destacadas da leitura — `prestacao_contas` negada, mas saldo permitido pela visão — são a demonstração prática do `SQL SECURITY DEFINER` descrito acima: o mesmo dado é liberado no agregado e negado na linha.

Senhas com hash e sal

Algoritmo: PBKDF2-HMAC-SHA256, 260.000 iterações, sal aleatório de 16 bytes por usuário.

Por que não hash simples. MD5, SHA-1 e SHA-256 puro foram feitos para integridade e são rápidos de propósito — justamente o que não se quer em senha. PBKDF2 é deliberadamente lento: o custo é irrelevante para um login legítimo e multiplica por centenas de milhares o custo de quem tenta adivinhar senhas em massa.

Por que sal por usuário. O sal não é segredo — fica ao lado do hash no banco. Sua função é garantir que duas pessoas com a mesma senha tenham hashes diferentes, o que inutiliza tabelas pré-calculadas e impede descobrir senhas repetidas apenas comparando colunas. O seed demonstra o efeito: a mesma senha derivada duas vezes produz hashes completamente diferentes.

Comparação em tempo constante. `hmac.compare_digest`, que gasta o mesmo tempo independentemente de onde os bytes diferem. Comparação comum com `==` vaza, pelo tempo de resposta, quantos bytes iniciais estavam corretos.

Na tela de login, senha errada e login inexistente devolvem a **mesma** mensagem: dizer qual dos dois falhou entregaria a um atacante a lista de logins válidos.

Dados pessoais

Dados que seriam considerados pessoais neste projeto, na acepção da LGPD:

Dado	Onde	Cuidado adotado
CPF	`associado_pf.cpf`	Nenhum perfil da aplicação lê a coluna; acesso só pela visão mascarada
Data de nascimento	`associado_pf.data_nascimento`	Mesma restrição de perfil
Nome civil	`associado.nome`	Visível para os três perfis, por ser necessário à operação
CNPJ e razão social	`associado_pj`	Mascarado na visão; dado de empresa, não de pessoa natural
Senha	`usuario_app`	Nunca armazenada; só o derivado PBKDF2 com sal

A carga usa **dados fictícios**, conforme o enunciado exige. Nenhum CPF, CNPJ ou nome corresponde a pessoa real.

Ressalva honesta: a aplicação não implementa registro de acesso a dado pessoal (quem consultou o quê), que seria exigível num sistema real. A auditoria implementada cobre alteração, não leitura.

---

7. B6 — Gatilhos e auditoria

A tabela de auditoria

Guardamos **uma linha por campo alterado**, não uma por `UPDATE`. Custa mais linhas, mas responde diretamente à pergunta que a cooperativa faz numa conferência: *"quem mudou o preço desta entrega,*

de quanto para quanto?". Com um único registro por operação seria preciso comparar dois blocos de valores para descobrir qual campo mudou.

Colunas: tabela, operação, chave do registro, campo, valor anterior, valor novo, usuário, data e hora.

Gatilho 1 — `entrega` (AFTER UPDATE)

Justificativa pelo domínio. A entrega é o documento que converte produção em dinheiro. Quantidade, preço e status definem quanto o associado recebe na prestação de contas. Uma alteração silenciosa em qualquer um dos três muda o valor a pagar sem deixar rastro. `qualidade` e `data_entrega` também são auditadas, porque reclassificar a qualidade de uma entrega é a forma mais discreta de alterar o preço pactuado.

Gatilho 2 — `prestacao_contas` (AFTER UPDATE)

Justificativa pelo domínio. É o fechamento financeiro, o número que o associado recebe. O `CHECK ck_prestacao_saldo` garante a coerência aritmética, mas **não impede** que alguém aumente desconto e reduza saldo de forma coerente e silenciosa. O gatilho cobre exatamente essa lacuna: a restrição protege a consistência, a auditoria protege a rastreabilidade.

Sobre `SESSION_USER()` e `CURRENT_USER()`

A coluna `usuario` grava `SESSION_USER()`, e não `CURRENT_USER()`.

A diferença aparece quando a rotina é executada com privilégio de outro: `CURRENT_USER()` devolve o **dono** da rotina — quem a definiu — enquanto `SESSION_USER()` devolve quem efetivamente **se conectou**. Para auditoria, o segundo é o correto: a pergunta que a trilha precisa responder é "quem fez", e não "sob qual privilégio rodou". Registrar o definidor transformaria a trilha num carimbo constante, inútil para apurar responsabilidade.

Além dos dois gatilhos de auditoria, a Etapa 2 acrescentou **sete** gatilhos de regra de negócio, descritos na seção 1 (resposta a C3). O banco tem hoje nove gatilhos ativos.

---

8. B7 — Projeto físico e armazenamento

Revisão dos tipos de dados

Coluna	Hoje	O que mudaríamos	Por quê
`associado.tipo_pessoa`	`CHAR(2)` + CHECK	manteria	2 caracteres fixos; `ENUM` economizaria pouco e dificultaria portar para PostgreSQL
`*.status` (várias)	`VARCHAR(20/30)` + CHECK	**tabela de domínio com FK**	o mesmo conjunto de valores está repetido em cinco CHECKs; acrescentar um status novo hoje exige `ALTER TABLE` em produção
`entrega.qualidade`	`CHAR(1)` + CHECK	manteria	domínio realmente fixo (A, B, C) e estável
`associado_pf.cpf`	`CHAR(11)`	manteria	tamanho fixo, sem máscara — decisão correta da Etapa 1
`prestacao_contas.saldo`	`DECIMAL(14,2)`	manteria	`DECIMAL` é obrigatório para dinheiro; ver o erro de `float` relatado em B3
`entrega.quantidade`	`DECIMAL(12,2)`	manteria	mesma razão
`auditoria.valor_anterior/novo`	`VARCHAR(255)`	`TEXT` se surgirem campos longos	hoje nenhuma coluna auditada passa de 255
chaves substitutas	`INT UNSIGNED`	`BIGINT UNSIGNED` em `entrega`	ver estimativa de crescimento abaixo

Estimativa de crescimento em dois anos

Medição real: `entrega` ocupa **195 bytes por linha** (dados + índices), com 200.200 linhas em 37,09 MB.

Premissas, que a cooperativa deveria revisar com dados próprios:

- 40 associados, 3 lotes por associado por safra, 2 safras por ano;
- 15 entregas por lote por safra → 40 × 3 × 15 × 2 = **3.600 entregas por ano**;
- crescimento de 20% ao ano no quadro de associados.

Tabela	Linhas hoje	Projeção em 2 anos	Tamanho projetado
`entrega`	160 reais	~7.900	~1,5 MB
`movimento_financeiro`	161	~1.000	~0,4 MB
`prestacao_contas`	81	~200	~0,1 MB
`auditoria`	9	~5.000	~2 MB
cadastros	~400	~600	desprezível

Conclusão honesta: no volume real deste domínio, o banco não chega a 10 MB em dois anos. `INT UNSIGNED` como chave é mais que suficiente — a troca para `BIGINT` só se justificaria se a cooperativa crescesse uma ordem de grandeza. A tabela que cresce mais rápido é a de **auditoria**, não a de operação, porque guarda uma linha por campo alterado.

Plano de backup

Item	Definição
O que copiar	Dump lógico completo do esquema `cooperativa_agricola`, incluindo rotinas e gatilhos (`mysqldump --routines --triggers --single-transaction`)
Frequência	Completo semanal (domingo, 2h); incremental diário via *binary log*
Retenção	4 semanas de completos, 7 dias de binlogs, 1 completo mensal por 12 meses
Onde	Cópia local em disco separado + cópia externa (nuvem ou mídia removível fora do prédio)
Teste de restauração	**Trimestral**, em instância limpa: restaurar o dump, executar as 15 consultas da Etapa 1 e conferir as contagens por tabela contra o relatório do dia do backup

A escolha de `--single-transaction` é deliberada: com InnoDB, ela obtém um dump consistente sem travar as tabelas, permitindo que o pátio continue operando durante a janela.

Por que o teste de restauração importa mais que o backup. Backup que nunca foi restaurado é hipótese, não garantia. O procedimento acima define o critério de sucesso — as contagens batem e as consultas rodam — e não apenas "o arquivo existe".

Particionamento e arquivamento

Nenhuma tabela justifica particionamento no volume real projetado. Se a cooperativa crescesse ao ponto de `entrega` passar de alguns milhões de linhas, a estratégia natural seria **particionar por `RANGE` sobre o ano de `data_entrega`**, porque as consultas do pátio sempre filtram por período — o otimizador poderia descartar partições inteiras. Safras encerradas há mais de cinco anos poderiam migrar para uma tabela de arquivo histórico.

Registramos a proposta sem implementá-la: particionar 160 linhas reais seria complexidade sem contrapartida.

---

9. Etapa 1 revisada

9.1 Dependências funcionais e verificação de BCNF

Atende à devolutiva C2: "demonstre a BCNF com as DFs (não apenas afirme)".

Notação: `X → Y` significa que X determina funcionalmente Y.

`associado`

```
id_associado → matricula, nome, tipo_pessoa, status, id_associado_indicador
matricula    → id_associado
```

Dois determinantes, ambos chaves candidatas (`id_associado` é PK, `matricula` tem UNIQUE). **BCNF**: todo determinante é chave candidata.

`associado_pf`

```
id_associado → cpf, data_nascimento
cpf          → id_associado
```

Ambos determinantes são chaves candidatas. **BCNF**.

`associado_pj`

```
id_associado → cnpj, razao_social
cnpj         → id_associado
```

BCNF, mesma estrutura.

`propriedade`

```
id_propriedade → id_associado, nome, registro_rural, municipio, area_hectares, status
registro_rural → id_propriedade
```

BCNF. Observação: `municipio` não determina nada aqui — não há `municipio → uf`, porque UF não é modelada. Se fosse, haveria DF transitiva e a 3FN exigiria tabela separada de municípios.

`cultura`

```
id_cultura → nome, categoria, unidade_medida
nome       → id_cultura
```

BCNF. `categoria → unidade_medida` **não** vale: duas culturas da mesma categoria podem ter unidades diferentes.

`safra`

```
id_safra  → descricao, data_inicio, data_fim, status
descricao → id_safra
```

BCNF.

`associado_cultura`

```
(id_associado, id_cultura) → area_hectares, data_inicio
```

Único determinante é a chave composta. **BCNF**.

`safra_cultura`

```
(id_safra, id_cultura) → meta_kg, preco_base_kg
```

BCNF. Note que `preco_base_kg` depende das duas colunas: o preço-base é da cultura *naquela safra*. Se dependesse só da cultura, haveria dependência parcial e violação de 2FN.

`lote` (entidade fraca)

```
(id_safra, numero_lote) → id_propriedade, id_cultura, quantidade_planejada, status
```

BCNF. `numero_lote` sozinho não determina nada — é por isso que `lote` é fraca.

`historico_associado`

```
id_historico              → id_associado, data_inicio, data_fim, situacao, motivo
(id_associado, data_inicio) → id_historico
```

Dois determinantes, ambos chaves candidatas (o segundo por `uq_historico_associado_inicio`). **BCNF**.

`entrega`

```
id_entrega → id_safra, numero_lote, data_entrega, quantidade, qualidade,
             preco_unitario_kg, status
```

BCNF. `qualidade → preco_unitario_kg` **não** vale: o preço é negociado por entrega, não tabelado por classificação. Se a cooperativa passasse a tabelar preço por qualidade, surgiria DF transitiva e seria preciso extrair uma tabela de preços.

`prestacao_contas`

```
id_prestacao              → id_associado, id_safra, total_bruto, total_descontos, saldo, status
(id_associado, id_safra)  → id_prestacao
total_bruto, total_descontos → saldo      ← DF entre atributos não-chave
```

A terceira DF **viola BCNF**, porque `(total_bruto, total_descontos)` não é chave candidata. É a única violação do esquema, e é **deliberada** — discutida a seguir.

`movimento_financeiro`

```
id_movimento → id_prestacao, data_movimento, valor, tipo, status, referencia
referencia   → id_movimento
```

BCNF.

9.2 O saldo derivado é redundância deliberada

Atende à devolutiva C2: "discuta se o saldo derivado é uma redundância deliberada".

Sim, e assumimos a escolha.

`saldo` é funcionalmente derivável de `total_bruto - total_descontos`. Mantê-lo armazenado viola BCNF, como a seção anterior mostra. A alternativa normalizada seria calcular na consulta ou expor por coluna gerada.

O que ganhamos: o saldo é o número que o associado vê e que a cooperativa usa em conciliação com o banco. Tê-lo materializado significa que ele pode ser indexado, somado por safra sem recalcular, e — principalmente — que fica **registrado o valor efetivamente apurado no fechamento**, não o valor que a fórmula produziria hoje. Se a regra de desconto mudar no futuro, o histórico não se reescreve sozinho.

O que pagamos: risco de divergência entre o armazenado e o derivável.

Como o risco é neutralizado: pela restrição `ck_prestacao_saldo CHECK (saldo = total_bruto - total_descontos)`, que torna a divergência impossível de gravar. A redundância existe, mas é uma redundância *verificada pelo banco*, não confiada à disciplina de quem escreve.

Foi justamente esse CHECK que denunciou o erro de `float` relatado na seção 4 — a redundância controlada funcionou como rede de segurança.

Alternativa que consideramos e descartamos: coluna gerada (`saldo DECIMAL(14,2) AS (total_bruto - total_descontos) STORED`). Ela eliminaria a violação de BCNF e manteria a materialização. Descartamos porque coluna gerada é sempre recalculada pela fórmula vigente, o que anularia o principal benefício: preservar o valor histórico apurado. Em um sistema de fechamento financeiro, o registro do que *foi* apurado vale mais que a elegância formal.

9.3 Estratégia PF/PJ comparada às alternativas

Atende à devolutiva C2: "compare também, por escrito, a estratégia de PF/PJ com as alternativas".

A Etapa 1 adotou **supertipo com duas tabelas de especialização** (`associado` + `associado_pf` + `associado_pj`). As três estratégias possíveis:

Alternativa A — tabela única com colunas nulas. Uma só tabela `associado` com `cpf`, `data_nascimento`, `cnpj` e `razao_social`, preenchendo o que couber.

- *A favor:* consulta mais simples, sem junção; menos tabelas.
- *Contra:* metade das colunas sempre nula, o que desperdiça espaço e, pior, **impossibilita** `NOT NULL` nos campos que são obrigatórios para cada tipo. A regra "todo PF tem CPF" deixaria de ser expressável como restrição de coluna e viraria CHECK condicional. O UNIQUE de CPF ainda funcionaria, mas a semântica fica frouxa.

Alternativa B — duas tabelas independentes, sem supertipo. `associado_pf` e `associado_pj` completas e separadas.

- *A favor:* cada tabela totalmente normalizada, sem nulos, sem junção.
- *Contra:* **inviabiliza a chave estrangeira única**. `propriedade.id_associado` precisaria apontar para uma das duas tabelas, e o modelo relacional não tem FK polimórfica. Seria preciso duas colunas de FK mutuamente exclusivas em `propriedade`, `prestacao_contas` e `historico_associado` — o

problema se espalha por todo o esquema. Além disso, matrícula única entre os dois tipos deixaria de ser garantível por UNIQUE.

Alternativa C — a adotada: supertipo + especializações.

- *A favor:* o supertipo concentra o que é comum (matrícula, nome, status, indicação) e serve de alvo único para todas as FKs do esquema; cada especialização tem seus campos `NOT NULL` de verdade; `uq_associado_matricula` vale para os dois tipos de uma vez.
- *Contra:* exige junção para montar a visão completa do associado, e **a totalidade e a disjunção não são garantidas pelo modelo** — exatamente o que a devolutiva C3 apontou.

Por que C continua sendo a escolha certa. O custo de C é uma junção a mais e duas regras que precisam de gatilho. O custo de B é uma FK polimórfica impossível, que contaminaria quatro tabelas. O custo de A é perder `NOT NULL` nos campos obrigatórios. A Etapa 2 pagou o custo de C implementando os gatilhos de disjunção e a visão de totalidade — e esse custo é local, confinado a uma tabela, enquanto os custos de A e B seriam estruturais.

9.4 Justificativa das ações referenciais

Atende à devolutiva C3: "justifique cada ação referencial".

FK	ON DELETE	ON UPDATE	Por quê
`fk_associado_indicador`	`SET NULL`	`CASCADE`	Se o indicador deixa a cooperativa, o indicado permanece — a indicação é um fato histórico, não uma dependência existencial. **Custo descoberto na Etapa 2:** impede `CHECK` na coluna (erro 3823)
`fk_associado_pf_associado`	`CASCADE`	`CASCADE`	A especialização não existe sem o supertipo; apagar o associado deve levar seus dados de PF
`fk_associado_pj_associado`	`CASCADE`	`CASCADE`	Mesma razão
`fk_propriedade_associado`	`RESTRICT`	`CASCADE`	Apagar associado com propriedades apagaria área produtiva e, em cascata, lotes e entregas. Exige transferir a propriedade antes — é a proteção que a aplicação transforma em mensagem ao usuário
`fk_associado_cultura_*`	`CASCADE` / `RESTRICT`	`CASCADE`	Cascata no associado (o vínculo não sobrevive a ele); restrição na cultura (não se apaga cultura em uso)
`fk_safra_cultura_safra`	`CASCADE`	`CASCADE`	O planejamento da safra morre com ela
`fk_safra_cultura_cultura`	`RESTRICT`	`CASCADE`	Não se apaga cultura com meta registrada
`fk_lote_safra`	`CASCADE`	`CASCADE`	`lote` é entidade fraca: sem safra não existe
`fk_lote_propriedade`	`RESTRICT`	`CASCADE`	Lote produzido é histórico; apagar a propriedade apagaria rastreabilidade
`fk_lote_cultura`	`RESTRICT`	`CASCADE`	Mesma razão
`fk_historico_associado`	`CASCADE`	`CASCADE`	O histórico é do associado e não faz sentido sem ele
`fk_entrega_lote`	`RESTRICT`	`CASCADE`	**A mais importante:** entrega origina pagamento. Nunca deve sumir por efeito colateral
`fk_prestacao_associado`	`RESTRICT`	`CASCADE`	Fechamento financeiro não se apaga em cascata
`fk_prestacao_safra`	`RESTRICT`	`CASCADE`	Mesma razão
`fk_movimento_prestacao`	`CASCADE`	`CASCADE`	O lançamento pertence à prestação; se ela for removida (o que `RESTRICT` acima torna raro), os lançamentos vão junto

O critério aplicado: `CASCADE` onde há **dependência existencial** (a linha filha não faz sentido sozinha); `RESTRICT` onde a linha filha é **registro histórico ou financeiro** que precisa sobreviver a enganos; `SET NULL` apenas onde o vínculo é opcional por natureza. `ON UPDATE CASCADE` em todas porque as chaves são substitutas e não mudam na prática — a cláusula é seguro barato.

---

10. Declaração de uso de inteligência artificial

Conforme o item 9 do enunciado, a equipe declara o uso de assistente de IA e como o resultado foi verificado.

Onde foi usada:

- redação e organização desta documentação e do README;
- escrita inicial do código da aplicação Flask, da camada de acesso a dados e dos *templates*;
- escrita inicial dos scripts SQL da Etapa 2 (visões, gatilhos, perfis, índices);
- construção dos scripts de experimento (teste de ponta a ponta, concorrência, injeção, dicionário).

Como o resultado foi verificado. Nenhuma afirmação deste relatório vem de sugestão aceita sem execução. Especificamente:

- os **36 testes** de aplicação, os **11** de perfil e os **10** de restrição foram executados contra o banco real, e a saída está em `experimentos/resultados/`;
- os números de desempenho do B4 são medições reais, com `EXPLAIN` e `EXPLAIN ANALYZE` capturados antes e depois de cada índice;
- o experimento de concorrência foi executado com duas conexões simultâneas, e o deadlock do cenário 3 é um erro 1213 real devolvido pelo servidor;
- o dicionário de dados é extraído do `information_schema`, não redigido.

Erros cometidos no processo, que ficaram registrados em vez de apagados:

1. a previsão da equipe sobre o índice da Consulta 2 (seção 5) estava **errada**, e o texto foi corrigido pela medição, não o contrário;
2. a primeira versão da transação usava `float` e foi reprovada pelo `CHECK` do banco (seção 4);
3. a tentativa de impor a autoindicação por `CHECK` falhou com erro 3823 e exigiu mudar para gatilho (seção 1);
4. o primeiro teste de cronologia do histórico tinha **expectativa errada** — foi o teste que estava equivocado, não o gatilho (seção 1);
5. e o mais relevante: o `02_carga.sql` da Etapa 1 **não executava em banco limpo**, defeito que só apareceu quando a equipe finalmente apagou o banco e reconstruiu do zero (seção 1, critério C4). Ele estava na entrega da Etapa 1 e passou despercebido por todas as verificações, porque todas rodaram sobre um banco já povoado.

A lição que levamos desta etapa, e que vale mais que qualquer artefato entregue: *verificação que roda sobre o ambiente de desenvolvimento não é verificação*. Os quatro primeiros erros acima foram encontrados porque executamos de verdade — contra o banco, com duas sessões, medindo tempo. O quinto foi encontrado porque, desta vez, apagamos tudo antes de testar. É por isso que o checklist do Anexo B começa com "o banco foi apagado": o item não é burocracia, é o único jeito de descobrir esse tipo de defeito.

Responsabilidade. A equipe revisou, executou e validou o conteúdo, e responde pelas decisões do projeto.